Parallelisation and SQL

FDE Tutorial Sessions, Week 4

Disclaimer

These slides are supplementary materials provided by the tutor and are not endorsed by the teaching team. They are intended to summarize the key concepts, provide pointers to additional resources and make the tutorial sessions easier to follow for students who are unfamiliar with the topic.

Please note that, in the event of any differences, the official lecture slides, solution sheets and other materials uploaded to Moodle should take precedence, particularly when preparing for exams.

Bits & SWAR

Bits, Hex and Masks

1 hex digit = 4 bits

1 byte = 2 hex digits

hex	*binary*	*decimal*
`0x0A`	`0000 1010`	`10 = '\n'`
`0x7F`	`0111 1111`	`127`
`0x80`	`1000 0000`	`128`
`0xFF`	`1111 1111`	`255`

ASCII characters are all ≤ 127 → highest bit 0

Where the mask (`m`) has 1:

`x & m` = and, keep the bit (others are cleared)

`x | m` = or, set the bit to 1

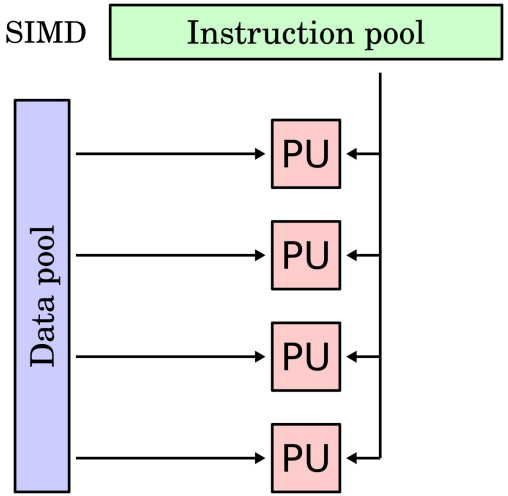
`x ^ m` = xor, flip the bit

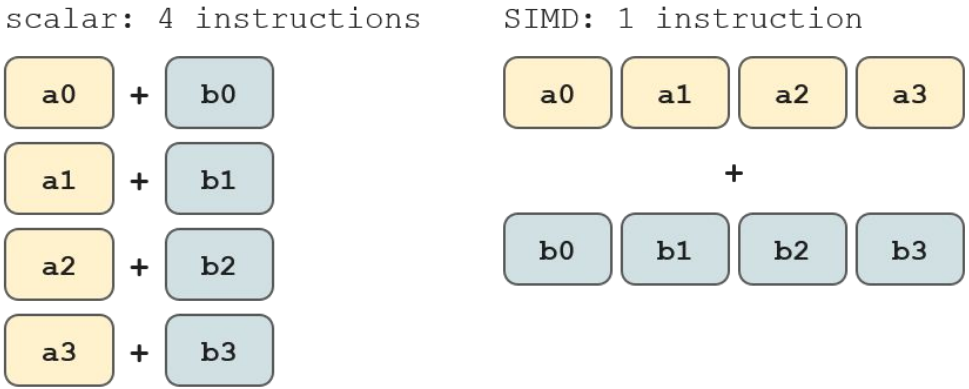
`~x` = not, flip every bit

Src: https://en.wikipedia.org/wiki/Bitwise_operation, https://en.wikipedia.org/wiki/ASCII

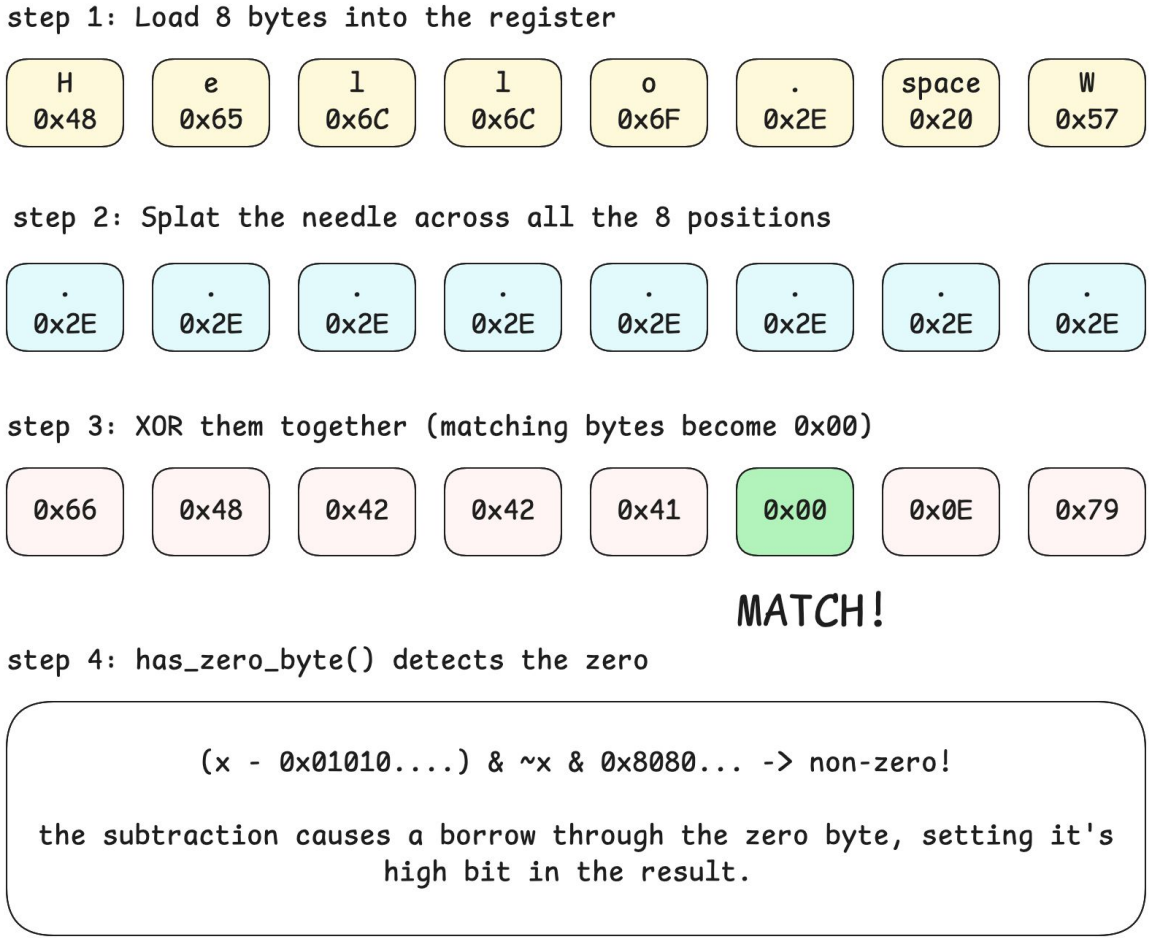
SWAR

SWAR = SIMD (=*single instruction, multiple data*) within a register





Img src: https://commons.wikimedia.org/wiki/File:SIMD.svg#/media/File:SIMD.svg; https://loran.hashnode.dev/vector-registers-simd-swarvector-chaining-any-difference

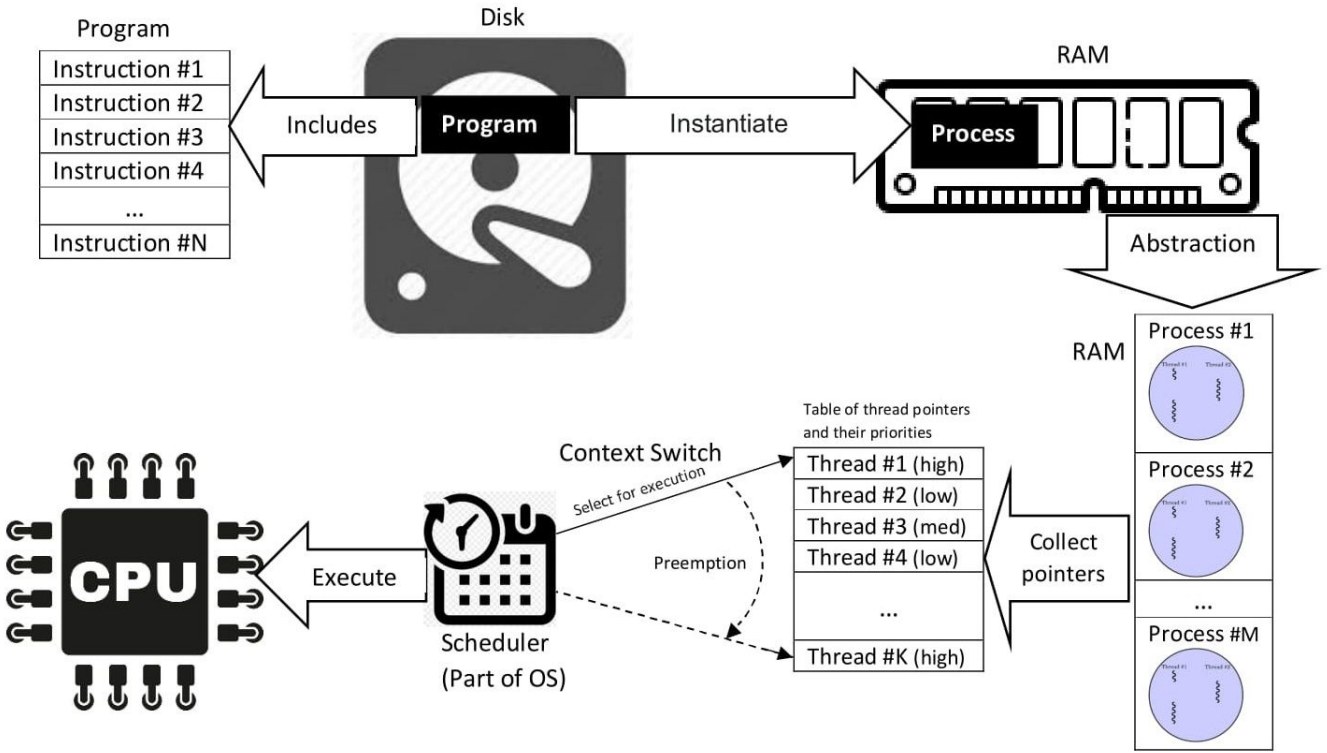


Src: https://minha.sh/posts/so%2C-you-want-to-chunk-really-fast

Attendance sheet

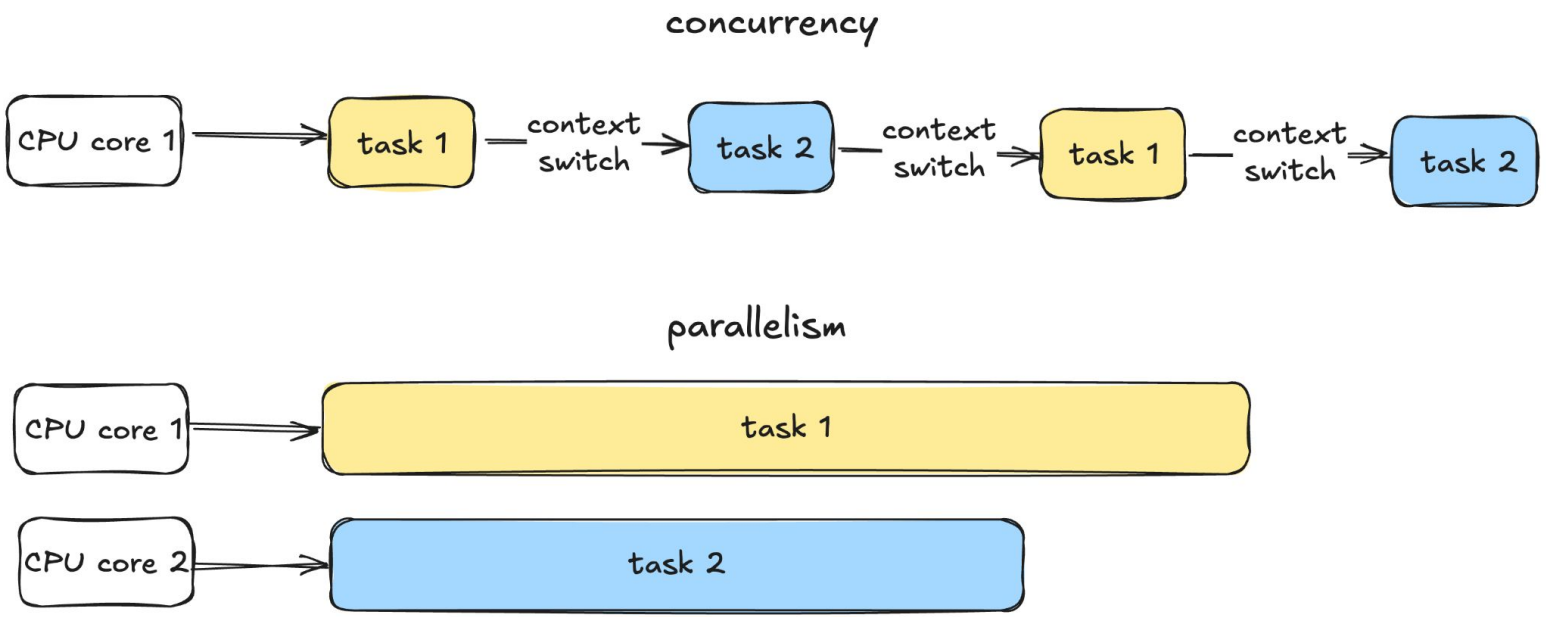
Threads

Program Organization



Src: https://commons.wikimedia.org/wiki/File:Concepts-_Program_vs._Process_vs._Thread.jpg#/media/File:Concepts-_Program_vs._Process_vs._Thread.jpg

Concurrency VS Parallelism



Img src: https://towardsdatascience.com/deep-dive-into-multithreading-multiprocessing-and-asyncio-94fdbe0c91f0/

std::thread

`std::thread::hardware_concurrency()` can tell you *approximately* how many threads your machine can support

A thread runs a function at the same time as the rest of the program

```
#include <thread>
#include <vector>
void work(int id) { /* … */ } // some function
int main() {
  std::vector<std::thread> threads;
  for (int i = 0; i < 4; ++i)
    threads.emplace_back(work, i); // starts now
  for (std::thread& t : threads)
    t.join();                    // wait for it
}
```

the thread starts right after construction!

wait for all of the threads to finish with `join`

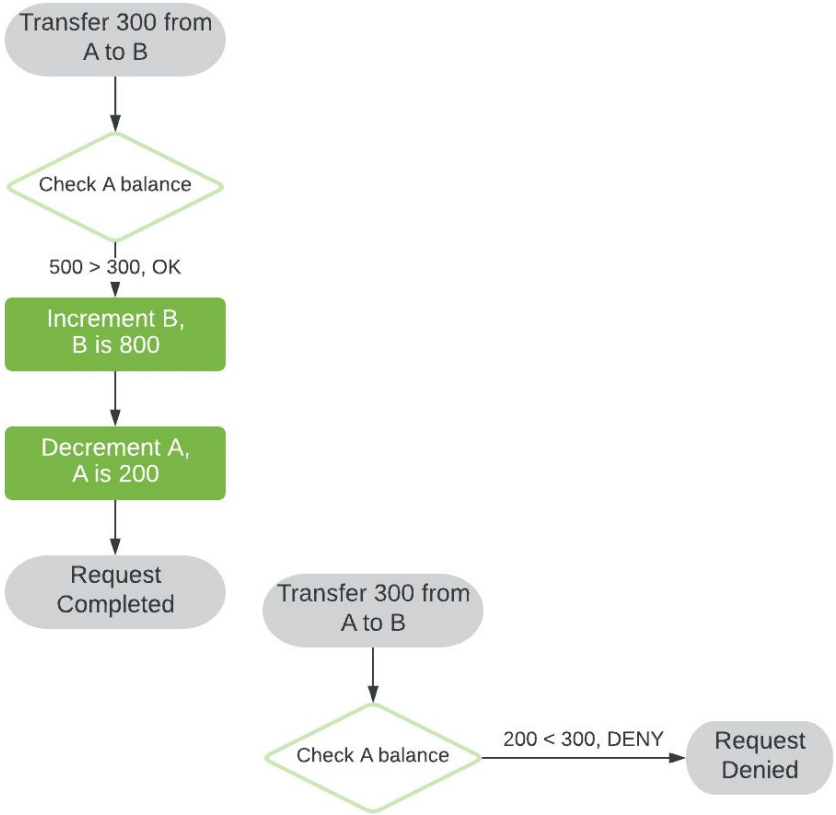
Src: https://minha.sh/posts/so%2C-you-want-to-chunk-really-fast

Race Conditions

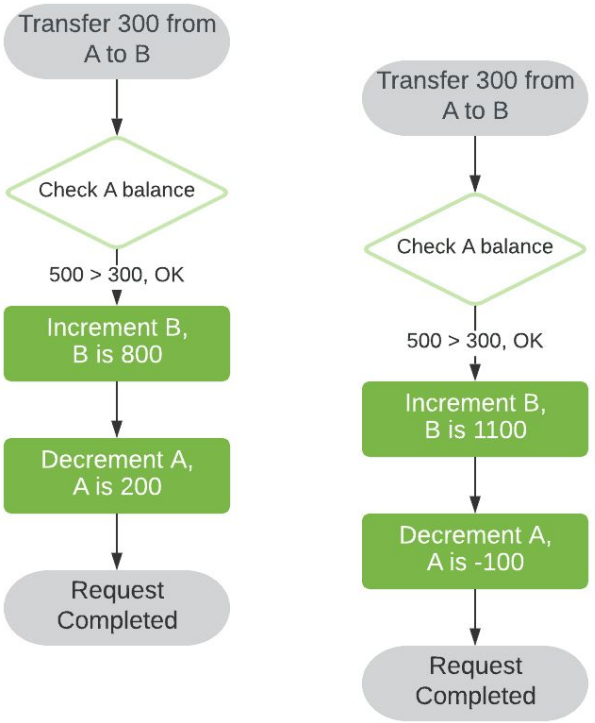
Race condition (race hazard) = the system's substantive behavior is **dependent on the sequence or timing of uncontrollable events**, leading to unexpected or **inconsistent** results. (Wikipedia)

Src: https://en.wikipedia.org/wiki/Race_condition

A.



B.



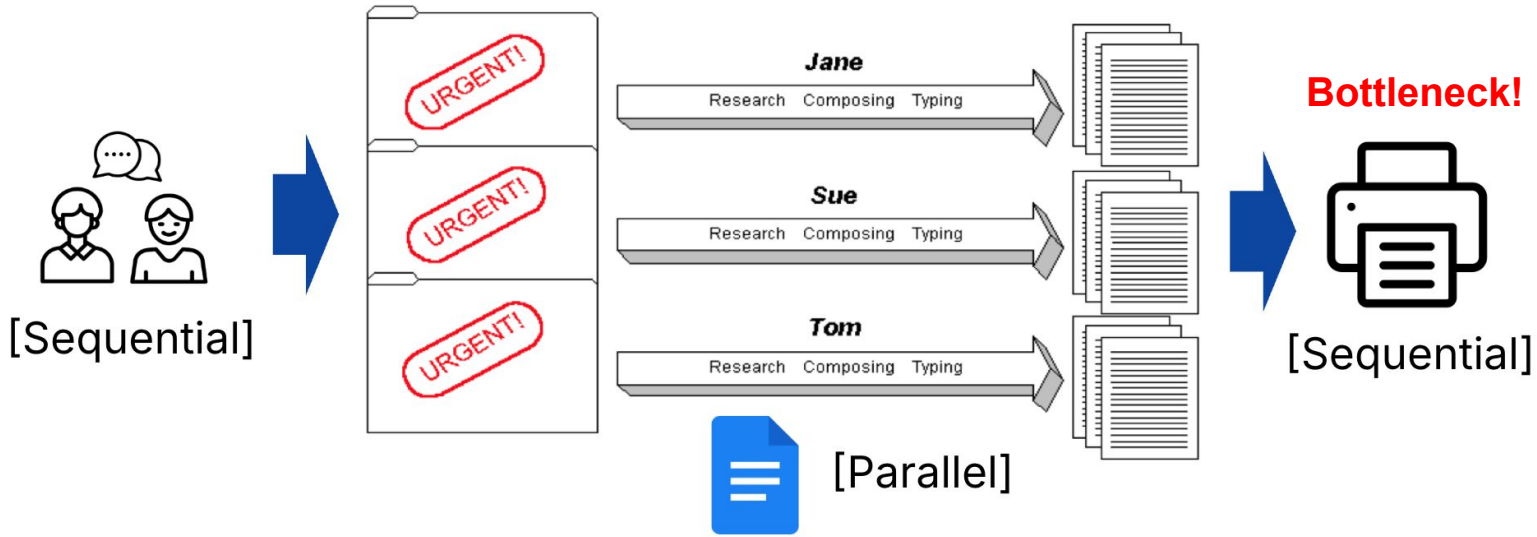
Src: https://www.baeldung.com/cs/race-conditions

Example

```
int counter = 0;
auto inc = [&] {
  for (int i = 0; i < 1'000'000; ++i) ++counter;
};
std::thread a(inc), b(inc);
a.join(); b.join();
std::cout << counter;  // expected: 2 000 000
```

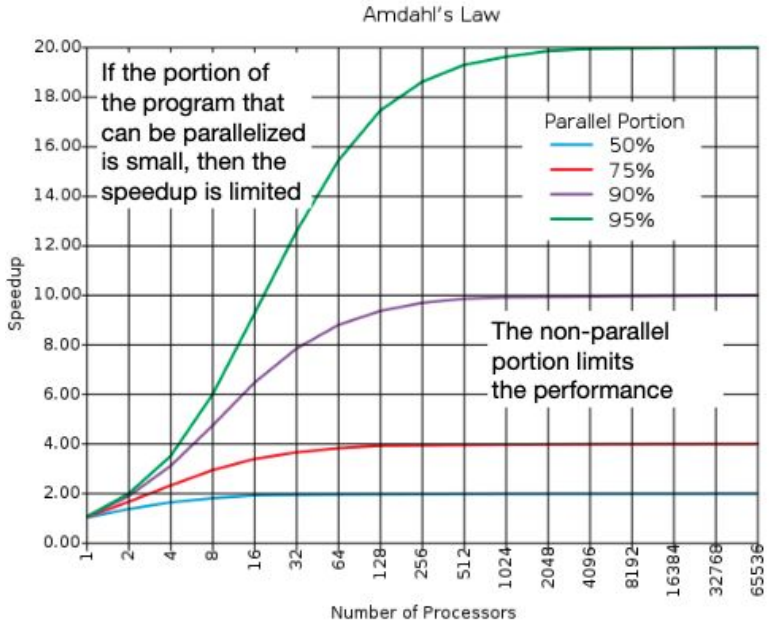
```
$ ./race  →  1 197 722
$ ./race  →  1 336 839
$ ./race  →  1 183 968
```

Amdahl's Law



Src: https://notes.cs61c.org/content/parallel-performance/

Amdahl's Law



$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

S(N) = the speedup of the program

P = the proportion of the program that can be parallelized (between 0 and 1)

N = the number of processors/cores

Src: https://notes.cs61c.org/content/parallel-performance/

SQL

SQL Query

```
SELECT l_returnflag, sum(l_quantity) AS qty
FROM lineitem
WHERE l_shipdate <= DATE '1998-09-02'
GROUP BY l_returnflag
HAVING sum(l_quantity) > 1000
ORDER BY qty DESC
LIMIT 10;
```

which columns / values to output

which table(s)

keep only some rows

one output row per group

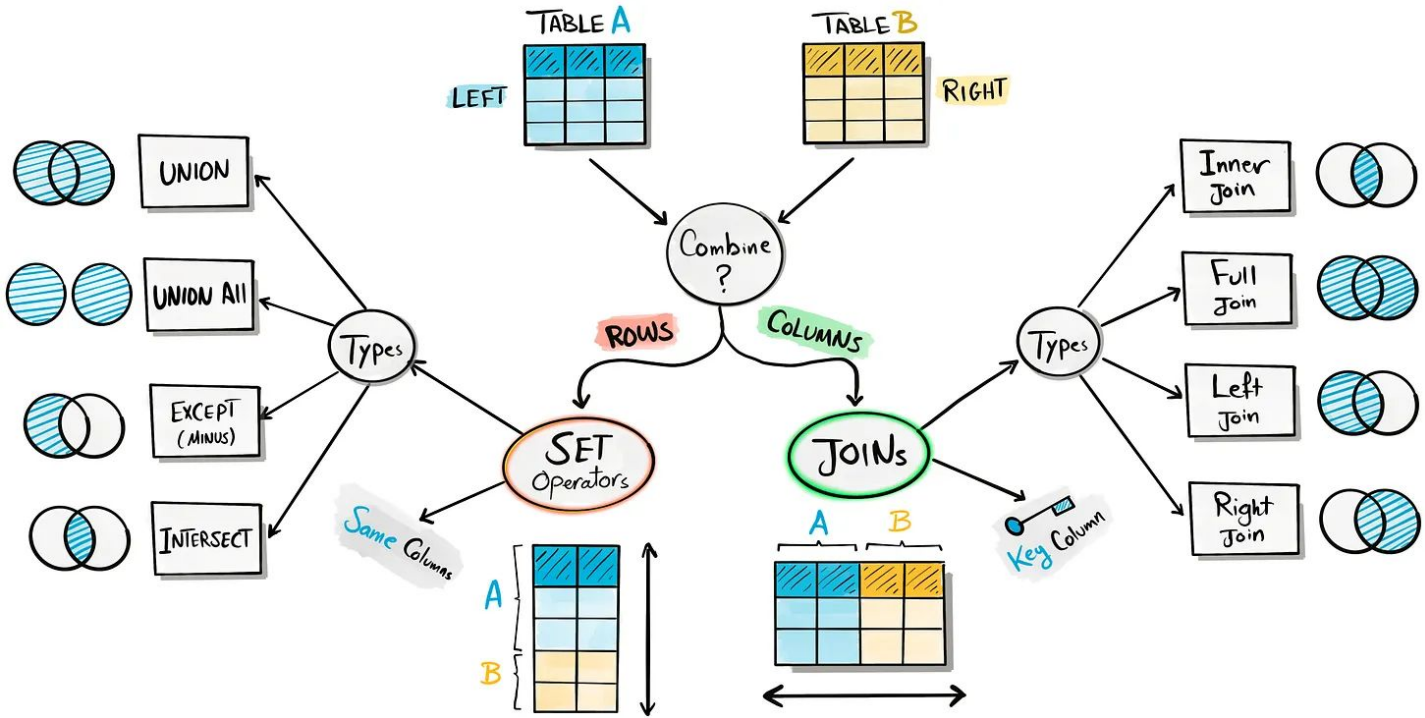
keep only some groups

sort the result

only the first rows

The order we write the query in is not the same as the order it is executed in!

Joins in SQL



Src: https://www.thedataschool.co.uk/le-luu/sql-joins/; Img src: https://www.blog.datawithbaraa.com/p/sql-joins-are-really-easy-and-fun

Query Execution



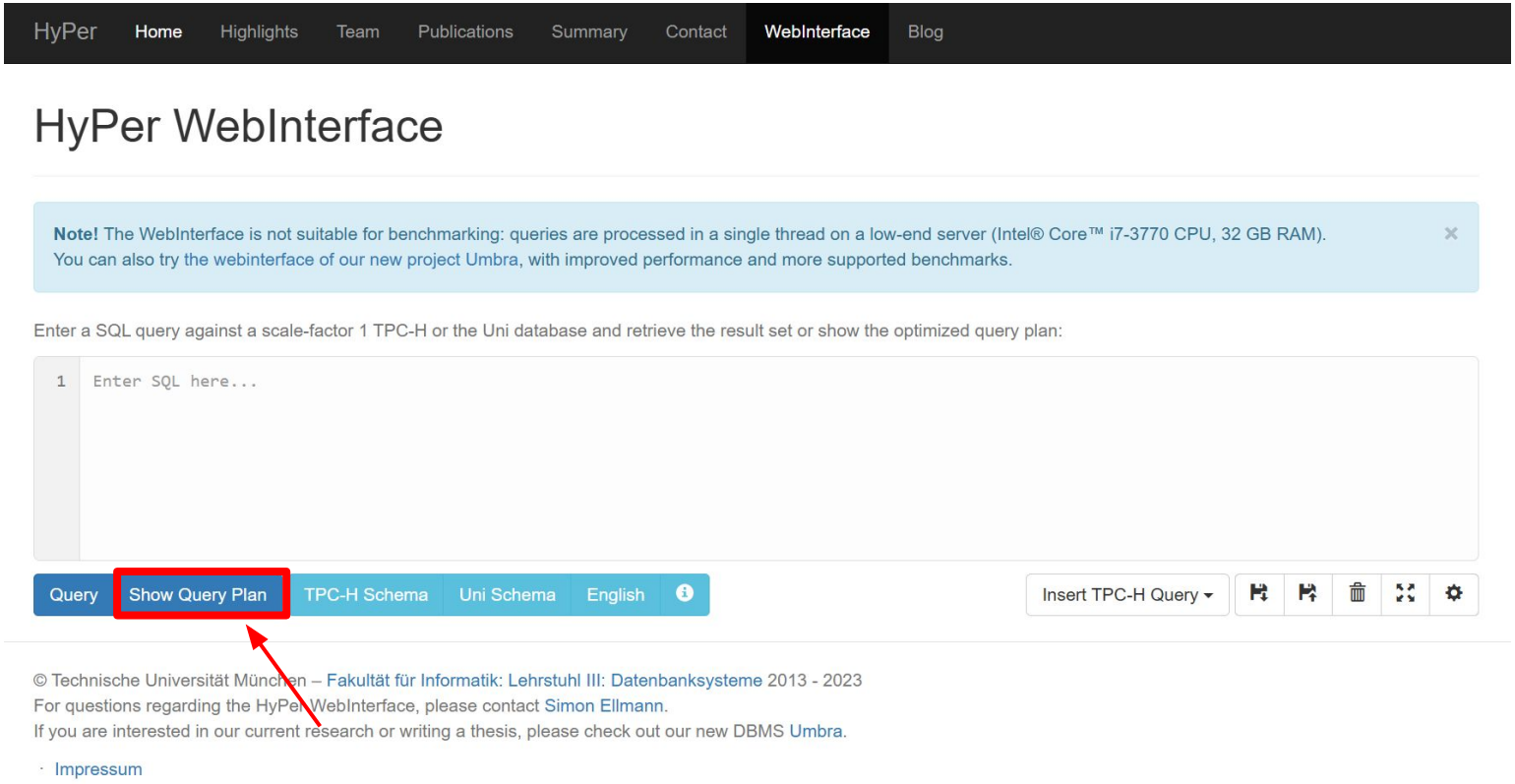
The optimizer decides:

- which **join algorithm** (nested loop? hash join?)
- which table to **build** on, in which **order** to join
- **where** to apply filters (as early as possible)

The result is the **query plan** = a tree of these operators

Src: https://db.in.tum.de/teaching/ws2627/queryopt/

HyPer Web Interface



Link: https://hyper-db.de/interface.html

Thank you!

Link to slides:

